APEX / PhysicsTTM — Wide-Pass Connective-Tissue Research Report

TL;DR

- **Build PhysicsTTM as three Python modules joined by one immutable tensor contract:** TTM emits `(B, H, C) = (1, 30, 14)` forecasts (the granite-tsfm source explicitly defines this shape: *"For a forecasting task, the shape is `(batch_size, target_len, num_input_channels)`"*); a differentiable physics-projection layer projects `(B, H, C) → (B, H, C)` onto a kinematically valid manifold using `cvxpylayers` for the convex core plus an optional 3-iteration unrolled SCP wrapper around a Pacejka linearization; and **IBM Granite Guardian 3.3** with the new `guardian_config = {"custom_criteria": ...}` API audits a JSON-derived natural-language violation log to issue a `<score>yes|no</score>` safety verdict. GitHub
- **For a 12-day, 2-person build, the realistic stable stack is:** FastAPI multipart upload → Pandas normalization to a fixed 14-channel canonical schema → polyphase 50→1Hz decomposition → TTM r2.1 (`prediction_filter_length=30` truncation) → `cvxpylayers` SOCP for the convex projection (friction circle + kinematic consistency + COA-gated complementarity) → optional 3-step unrolled SCP for tire nonlinearity → Plotly g-g and friction-ellipse overlays as the demo centerpiece. The non-convex Pacejka tier should live behind a feature flag; the convex projection alone is sufficient to demonstrate kinetic-hallucination mitigation.
- **The single most compelling demo asset is a side-by-side g-g diagram** showing the raw TTM forecast spilling outside the friction circle (with a "throttle at 0 / speed rising" annotated anomaly), versus the PhysicsTTM-projected forecast neatly bounded by the friction ellipse — overlaid with a Granite Guardian SAFE stamp generated from the auto-written violation log. The marquee story beat is the COA toggle: an MME/Team BRIT hand-control driver (whose product page reads verbatim *"Simultaneous control — Holding full throttle while shifting or holding full throttle and applying a little braking in the corner"*) versus a Sam-Schmidt-style sip-and-puff driver — same TTM, same telemetry, one boolean from the CoA changes the constraint set. Mme-motorsport

---

Key Findings

1. **TTM's forecast tensor shape is fixed, deterministic, and differentiable-friendly.** `TinyTimeMixerForPrediction(...).forward(past_values=...).prediction_outputs` returns `(batch_size, prediction_length, num_input_channels)`. The granite-tsfm source (`modeling_tinytimemixer.py`) states verbatim: *"For a forecasting task, the shape is*

`(batch_size, target_len, num_input_channels)`." The config field `prediction_filter_length=k` truncates the middle dimension via `future_values_masked[:, :self.config.prediction_filter_length, :]`. This gives a clean `(B, 30, 14)` tensor to project. GitHub

2. **TTM is channel-independent during pretraining, which is exactly the architecture that produces "kinetic hallucination."** IBM Research's own write-up states: *"TTM is first pretrained in a channel-independent way (univariate sequences) and uses cross-channel mixing during finetuning."* Since APEX uses TTM zero-shot (no time to fine-tune on motorsport data in 12 days), the physics-projection layer is the **only** mechanism enforcing cross-channel consistency. That is the architectural justification for the entire project. arXiv Towards Data Science
3. **Granite Guardian 3.3 (August 2025) ships a clean BYOC API that is ideal for physics-violation auditing.** Pass `guardian_config = {"custom_criteria": "<free-text criterion>"}` into `tokenizer.apply_chat_template(...)` along with the user/assistant messages; parse the response for `<score>yes|no</score>` and optional `<think>...</think>` traces. The August 2025 release notes state verbatim: *"New hybrid thinking mode for better reasoning and improved bring-your-own-criteria functionality."* PhysicsTTM's Layer C is therefore a one-shot call: hand Guardian the physics-violation log + the custom criterion *"The assistant message describes a kinematically valid vehicle trajectory consistent with the driver's certified control adaptations,"* and receive a binary safety stamp. github
4. **The FIA Certificate of Adaptations is parameterizable from a well-defined taxonomy of nine adaptation categories** documented in the FIA Disability and Accessibility Commission's *Vehicle Adaptation Guidelines, Edition 1, December 2022* (PDF on fia.com): (2.1) Throttle Management — including hand-lever subtypes Steering-Wheel-Backside, Steering-Wheel-Frontside, Hand Lever; (2.2) Brake Management; (2.3) Clutch System (electronic e-clutch, with explicit suppliers MEGAline and AP Racing named); (2.4) Steering Assembly (impact absorber, quick release, ⩾150 cm² impact surface); (2.5) Gearshift and Gearbox; (2.6) Seat and Driver Restraints; (2.7) Headrest and Cockpit; (2.8) Driver Equipment; (2.9) Chassis Modification. The single most physics-relevant rule is documented verbatim: *"Brake forces have to be considered while designing the 'hand throttle', to ensure that the throttle is not inadvertently activated during application of the brake."* This is precisely the kind of language that maps to a tensor-level toggle. fia fia
5. **The FIA-recognized Team BRIT / MME hand controls EXPLICITLY ALLOW simultaneous throttle and brake application.** MME Motorsport's product page (the sole manufacturer; the system is FIA-approved at ≈£7,500 per Adrian Flux's interview with Team BRIT principal Dave Player) states verbatim: *"Simultaneous control — Holding full throttle while shifting or holding full throttle and applying a little braking in the corner."* PMW Magazine confirms: *"The system is able to take inputs such as holding full throttle while shifting, or holding full throttle and applying a little braking mid-corner."* In contrast, Sam Schmidt's quadriplegic-driver setup (Adrian Flux): *"He accelerates and brakes by*

blowing into and sucking air from a tube" — physically complementary (you cannot fully inhale and exhale at the same time). **This is the critical finding:** the COA-gated complementarity constraint must be **disabled** for MME-style drivers and **enabled** for sip-and-puff drivers — and this is the most compelling demo case. Adrian Flux + 3

6. **`cvxpylayers` + a 3-iteration unrolled SCP wrapper is the right engineering compromise for 12 days.** `cvxpylayers` provides DPP-compliant convex QP/SOCP layers with implicit-differentiation backward passes; for the non-convex Pacejka tier, wrap a first-order Taylor-linearized tire model inside an outer Python loop (3 SCP iterations) and let PyTorch unroll the gradient. `qpth/OptNet` is ~5× faster than `cvxpylayers` on GPU per the OptNet paper, but is QP-only — no SOCP support, which kills the friction-circle constraint (`||(ax, ay)||₂ ≤ μ·g` is a second-order cone). `theseus` is the wrong tool — Meta's library is for differentiable nonlinear *least-squares* in SLAM/robotics, and its own NeurIPS 2022 paper admits it applies constraints *"in a soft manner (i.e., using weighted costs)"* — which defeats the entire point of hard physics constraints. The non-convex tier should be feature-flagged: if it is not stable by Day 9, ship the convex tier alone. GitHub + 5

Details

1. Integration Architecture — How PhysicsTTM Wires Together

1.1 The Canonical Tensor Contract

The system has exactly **one** internal tensor representation flowing between TTM, the projection layer, and Guardian's textual log generator. Define it once in `apex/contracts.py`:

```python
```

```python
# apex/contracts.py
from typing import Final

# Canonical channel ordering — frozen for the whole project.
# Union of native data-logger fields (AiM/MoTeC/FastF1) and computed channels.
# Missing channels are imputed from kinematics in preprocessing.
CHANNEL_ORDER: Final = (
    "time_s",        # 0  monotonic seconds since lap start (INDEX, not forecast)
    "speed_mps",     # 1  vehicle speed, m/s
    "ax_mps2",       # 2  longitudinal acceleration (vehicle frame), m/s^2
    "ay_mps2",       # 3  lateral acceleration (vehicle frame), m/s^2
    "throttle_norm", # 4  throttle pedal/lever, [0, 1]
    "brake_norm",    # 5  brake pedal/lever, [0, 1] (FLOAT, not bool — see Caveats)
    "steer_rad",     # 6  steering angle at wheel, radians (+ve = left)
    "yaw_rate_rps",  # 7  yaw rate, rad/s
    "rpm",           # 8  engine RPM (or motor RPM)
    "gear",          # 9  gear index, cast to float
    "pos_x_m",       # 10 GPS-derived track-frame X, meters
    "pos_y_m",       # 11 GPS-derived track-frame Y, meters
    "fz_total_n",    # 12 estimated total vertical load, N
    "mu_est",        # 13 estimated friction coefficient, dimensionless
)
NUM_CHANNELS: Final = 14
TTM_CONTEXT_LEN: Final = 512      # matches granite-timeseries-ttm-r2 "512_96" branc
TTM_PREDICTION_LEN: Final = 96    # native head length
PREDICTION_FILTER_LEN: Final = 30 # truncate via TTM's prediction_filter_length
NATIVE_HZ: Final = 50             # data-logger native rate
TTM_HZ: Final = 1                 # TTM operates at 1Hz aggregated rate
```

The AiM-CSV / MoTeC-CSV header remapping pattern (see goprotelemetryextractor's `remapheaders.txt` convention) lets a single CSV loader handle the wide variety of consumer/grassroots data-logger formats: stream identifiers like `LongitudinalAcceleration`, `LateralAcceleration`, `SteeringAngle`, `Throttle%`, `BrakePressure` are mapped onto the canonical channel ordering. Goprotelemetryextractor

1.2 The Forecast Tensor Shape

After polyphase 50→1Hz decomposition (covered in detail by Model 2's research pass), TTM input is `past_values ∈ ℝ^(B, 512, 14)`. `TinyTimeMixerForPrediction.forward(past_values=...).prediction_outputs` returns exactly `(B, 30, 14)` when constructed with `prediction_filter_length=30`. Verbatim from `modeling_tinytimemixer.py`: *"For a forecasting task, the shape is `(batch_size, target_len,`*

`num_input_channels)`. *Even if we want to forecast only specific channels by setting the indices in* `prediction_channel_indices` *parameter, pass the target data with all channels, as channel filtering for both prediction and target will be manually applied before the loss computation."* GitHub

The physics-projection layer receives this `(B, 30, 14)` tensor unchanged and emits a tensor of the same shape, projected onto the physically valid manifold. **No reshape, no permute, no channel renaming.** A developer can write `assert y.shape == (B, 30, 14)` after every module call.

1.3 The Three-Layer Pipeline as Python Modules

```
apex/
├── contracts.py                # CHANNEL_ORDER, Pydantic models, type aliases
├── io/
│   ├── csv_loader.py           # AiM/MoTeC/FastF1 CSVs → canonical 14-channel df
│   ├── coa_parser.py           # PDF → CoAParameters (Pydantic)
│   └── debrief_loader.py       # Plain text → list[CornerNote]
├── preprocess/
│   ├── polyphase.py            # 50Hz → 1Hz polyphase decomposition (per Model 2)
│   ├── imputation.py           # Fill missing channels via kinematic identities
│   └── normalize.py            # tsfm_public TimeSeriesPreprocessor wrapper
├── forecast/
│   └── ttm_runner.py           # Layer A: TinyTimeMixerForPrediction wrapper
├── physics/
│   ├── constraints.py          # Pure functions returning CVXPY constraint lists
│   ├── projection_layer.py     # Layer B: nn.Module wrapping CvxpyLayer + SCP
loop
│   ├── pacejka.py              # Differentiable Pacejka MF (optional, feature-
flag)
│   └── violation_logger.py     # Compares raw vs projected, emits structured log
├── safety/
│   └── guardian_runner.py      # Layer C: Granite Guardian 3.3 BYOC call
├── coach/
│   ├── corner_segmenter.py     # GPS + steering → per-corner segments
│   ├── tuning_recommender.py   # Heuristics over projected-forecast deltas
│   └── report_writer.py        # Markdown/HTML coaching report
└── api/
    └── main.py                 # FastAPI routes
```

1.4 The Violation Log Schema (Layer B → Layer C bridge)

Layer B compares `y_raw` to `y_proj` and emits a structured Pydantic log. This is the **only** interface between the physics layer and Guardian — Guardian sees text, never tensors.

python

```python
# apex/physics/violation_logger.py
from pydantic import BaseModel
from typing import Literal

class PhysicsViolation(BaseModel):
    t_index: int                                # forecast timestep (0..29)
    constraint: Literal[
        "friction_circle",
        "kinematic_speed_accel",                # d(speed)/dt vs ax
        "throttle_brake_complementarity",
        "steering_lateral_consistency",         # |ay| consistent with steer * v^2 / L
        "load_transfer_bounds",
        "tire_force_envelope",                  # Pacejka tier only
    ]
    severity: float                              # L2 of (y_raw - y_proj) on channels
    channels_affected: list[str]
    raw_values: dict[str, float]
    projected_values: dict[str, float]
    explanation: str                             # human-readable, fed to Guardian

class ViolationLog(BaseModel):
    driver_id: str
    coa_summary: str
    forecast_horizon_s: float
    n_violations: int
    max_severity: float
    violations: list[PhysicsViolation]

    def to_guardian_prompt(self) -> str:
        """Render as natural language for Granite Guardian's BYOC criterion."""
        ...
```

This schema is append-only, stable, and Guardian-friendly. The `explanation` field is concatenated into Guardian's prompt.

1.5 Granite Guardian 3.3 BYOC Call Pattern (verbatim)

The August 2025 model card for `ibm-granite/granite-guardian-3.3-8b` documents the BYOC pattern precisely:

```python
user_text = violation_log.to_guardian_prompt()
custom_criteria = (
    "The assistant message describes a kinematically valid vehicle "
    "trajectory consistent with the driver's certified control adaptations. "
    "All physics-correction events were caused by the forecasting model "
    "producing inputs that violate Newton's laws or the driver's hardware "
    "constraints, NOT by the corrected trajectory itself being unsafe."
)
messages = [{"role": "user", "content": user_text}]
guardian_config = {"custom_criteria": custom_criteria}

chat = tokenizer.apply_chat_template(
    messages,
    guardian_config=guardian_config,
    think=True,
    tokenize=False,
    add_generation_prompt=True,
)
output = model.generate(chat, sampling_params)
score, trace = parse_response(output[0].outputs[0].text.strip())
# score ∈ {"yes", "no"}; trace = <think>...</think> contents
```

For Granite Guardian 3.2-5B (the prior version), BYOC requires the older free-form template with explicit `<start_of_risk_definition>...<end_of_risk_definition>` tags — workable but messier. **Target 3.3** at submission time. huggingface

2. Frontend ↔ Backend Data Contract

2.1 The Three-File Upload Endpoint

FastAPI cannot mix `UploadFile` with a JSON Pydantic body in the same multipart request — a documented limitation (fastapi/fastapi#657: *"UploadFile does not seem to be compatible with Pydantic… ValueError: Value not declarable with JSON Schema, field: file"*). The clean pattern is `multipart/form-data` with all three files plus a JSON-stringified metadata form field: GitHub

```python
```

```python
# apex/api/main.py
from fastapi import FastAPI, UploadFile, File, Form, BackgroundTasks
from pydantic import BaseModel
from uuid import uuid4
from typing import Literal

app = FastAPI(title="APEX", version="0.1.0")

class SessionMetadata(BaseModel):
    driver_alias: str
    track_name: str
    car_class: str | None = None
    session_type: Literal["practice", "qualifying", "race", "test"] = "practice"

@app.post("/api/v1/session", status_code=202)
async def submit_session(
    telemetry: UploadFile = File(..., description="CSV from AiM/MoTeC/FastF1"),
    coa_pdf:   UploadFile = File(..., description="FIA Certificate of Adaptations"),
    debrief:   UploadFile = File(..., description="Plain-text driver debrief"),
    metadata_json: str = Form(..., description="JSON-stringified SessionMetadata"),
    background_tasks: BackgroundTasks = None,
):
    metadata = SessionMetadata.model_validate_json(metadata_json)
    job_id = str(uuid4())
    # Persist all three files to /tmp/apex/{job_id}/
    background_tasks.add_task(run_pipeline, job_id, metadata)
    return {"job_id": job_id, "status_url": f"/api/v1/session/{job_id}"}

@app.get("/api/v1/session/{job_id}")
async def poll_session(job_id: str) -> SessionStatus:
    """queued | running:preprocess | running:forecast | running:projection
       | running:safety | done | failed"""

@app.get("/api/v1/session/{job_id}/report")
async def get_report(job_id: str) -> CoachingReport: ...
```

This contract is the minimal surface that lets frontend and backend work independently. **Backend ships a stub on Day 1 that returns canned `CoachingReport` JSON; frontend integrates against the stub immediately.** Real work happens in `BackgroundTasks` so the upload returns in <1 s.

2.2 The Coaching Report Response Schema

python

```python
class CornerCoaching(BaseModel):
    corner_id: int
    corner_name: str | None
    distance_m_start: float
    distance_m_end: float
    raw_forecast_summary: dict[str, list[float]]      # subsampled ~10 pts/corner
    projected_forecast_summary: dict[str, list[float]]
    delta_lap_time_s: float                           # predicted gain if tuning ap
    coach_text: str                                   # 2-3 sentence narrative
    visualizations: list[str]                         # URLs

class TuningRecommendation(BaseModel):
    parameter: Literal["front_arb", "rear_arb", "brake_bias",
                       "diff_preload", "tire_pressure"]
    direction: Literal["increase", "decrease", "no_change"]
    magnitude_pct: float
    confidence: float
    rationale: str

class SafetyVerdict(BaseModel):
    verdict: Literal["SAFE", "REVIEW", "UNSAFE"]
    guardian_score: Literal["yes", "no"]              # raw Guardian output
    guardian_reasoning: str | None                    # <think>...</think> contents
    n_violations_corrected: int
    max_severity: float

class CoachingReport(BaseModel):
    job_id: str
    generated_at: datetime
    corners: list[CornerCoaching]
    tuning: list[TuningRecommendation]
    safety: SafetyVerdict
    next_session_forecast_uri: str                    # parquet URL
```

2.3 Intermediate File Formats

Persist between stages in `/tmp/apex/{job_id}/`:

Persisting tensors between stages is the difference between a debuggable system and an opaque one for a 12-day build. **NumPy `.npy` for tensors, Parquet for tabular, JSON for everything else.**

3. COA-Parameterization End-to-End

3.1 The Pydantic CoA Schema

The FIA Vehicle Adaptation Guidelines (December 2022) define nine categories. Each maps onto machine-readable fields that drive physics-constraint toggles:

```python
```

```python
# apex/io/coa_parser.py
from pydantic import BaseModel
from typing import Literal

class ThrottleAdaptation(BaseModel):
    type: Literal[
        "standard_pedal",
        "hand_lever_pull",
        "hand_lever_push",
        "steering_wheel_backside_paddle",     # FIA §2.1.3
        "steering_wheel_frontside_paddle",    # FIA §2.1.4
        "sip_puff",
        "joystick",
    ]
    is_electronic: bool = False               # fly-by-wire vs cable
    mechanically_coupled_to_brake: bool = False  # sip-puff = naturally complementar

class BrakeAdaptation(BaseModel):
    type: Literal[
        "standard_pedal", "hand_lever_pull", "hand_lever_push",
        "steering_wheel_paddle", "sip_puff",
    ]
    actuation: Literal["mechanical", "pneumatic", "hydraulic_assisted"] = "mechanica
    max_force_n: float | None = None

class HandControlSystem(BaseModel):
    permits_simultaneous_throttle_brake: bool   # THE physics-toggle field
    manufacturer: str | None = None              # "MME Motorsport" / "Bosch" / etc.
    fia_recognized: bool = True

class SteeringAdaptation(BaseModel):
    quick_release: bool = True
    reduced_lock_to_lock_deg: float | None = None

class CoAParameters(BaseModel):
    driver_classification: Literal["ambulant", "non_ambulant"]
    coa_reference: str | None
    issued_date: str | None
    throttle: ThrottleAdaptation
    brake: BrakeAdaptation
    hand_control: HandControlSystem | None
    steering: SteeringAdaptation
    clutch_type: Literal["standard", "electronic", "none_paddle_shift"]
```

```
    def to_physics_flags(self) -> "PhysicsConstraintFlags":
        return PhysicsConstraintFlags(
            enforce_throttle_brake_complementarity=(
                self.hand_control is None
                or not self.hand_control.permits_simultaneous_throttle_brake
            ),
            steering_max_rad=(
                self.steering.reduced_lock_to_lock_deg or 540
            ) * (3.14159 / 180.0) / 2,
            enforce_brake_force_limit=(self.brake.max_force_n is not None),
            brake_max_force_n=self.brake.max_force_n,
        )

class PhysicsConstraintFlags(BaseModel):
    enforce_throttle_brake_complementarity: bool = True
    enforce_steering_lock_limit: bool = True
    steering_max_rad: float = 4.71              # 270 deg one-side default
    enforce_brake_force_limit: bool = False
    brake_max_force_n: float | None = None
```

3.2 PDF → Pydantic via LLM Structured Extraction

With limited or no real example CoA documents, the pragmatic approach is two stages:

Stage A — Layout-aware text extraction. Use `pdfplumber` (rule-based) or `pymupdf4llm` (markdown output) to pull text. For scanned/OCR PDFs (likely with older CoAs), fall back to `pytesseract`. The Unstract benchmark on real-world PDFs makes a clear case that LLMWhisperer or PyMuPDF4LLM beat raw PyPDF for form-style documents with checkboxes — relevant here because CoAs have form-like adaptation tick-boxes.

Stage B — LLM-driven structured extraction. Use LangChain's `with_structured_output(CoAParameters)` (Pydantic-binding via tool-calling or JSON-mode) with an IBM Granite Instruct model: Zipstack Unstract

```
python
```

```python
from langchain_core.prompts import ChatPromptTemplate

EXTRACTION_PROMPT = ChatPromptTemplate.from_messages([
    ("system", """You extract structured vehicle-adaptation parameters from FIA
    Certificate of Adaptations documents. The FIA Disability and Accessibility
    Commission's Vehicle Adaptation Guidelines (Dec 2022) define nine categories:
    Throttle Management (§2.1), Brake Management (§2.2), Clutch System (§2.3),
    Steering Assembly (§2.4), Gearshift (§2.5), Seat/Restraints (§2.6),
    Headrest/Cockpit (§2.7), Driver Equipment (§2.8), Chassis (§2.9).

    Important physical realities:
    - MME Motorsport / Team BRIT hand-paddle controls PERMIT simultaneous
      throttle and brake (manufacturer states this verbatim).
    - Sip-and-puff controls (e.g. Sam Schmidt) CANNOT — inhalation and
      exhalation are mutually exclusive.
    - The FIA guideline states 'Brake forces have to be considered while
      designing the hand throttle, to ensure that the throttle is not
      inadvertently activated during application of the brake.'

    If a field is not mentioned, default to standard able-bodied configuration."""),
    ("user", "FIA COA document text:\n\n{document_text}\n\nExtract parameters."),
])

def parse_coa_pdf(pdf_path: str, llm) -> CoAParameters:
    text = extract_text_layout_aware(pdf_path)
    chain = EXTRACTION_PROMPT | llm.with_structured_output(CoAParameters)
    return chain.invoke({"document_text": text})
```

3.3 Handling Limited / No Example Documents

For the hackathon, build a **synthetic CoA corpus** of ~10 LaTeX-generated PDFs that mirror the FIA guideline section headings verbatim, covering the canonical cases:

Synthetic CoA	Hand-control type	`permits_simultaneous_throttle_brake`
1 — Wickens-style paddles	MME / Bosch electronic paddles	**True**
2 — Sam-Schmidt-style sip-puff	Pneumatic sip-puff	**False** (physically)
3 — Single-leg amputee (right)	Standard pedals, left foot only	**True** (left-foot-brake)
4 — Single-leg amputee (left)	Standard pedals, right foot only	**False** (one foot, two pedals)
5 — Reduced upper-limb mobility	Standard pedals, reduced steering lock	depends on pedal layout
6–10	Variations on above	...

This synthetic corpus serves three purposes: (a) testing the parser; (b) seeding the LLM's few-shot prompt; (c) demo evidence that the system handles diverse cases. **Build this on Day 3.**

3.4 Tensor-Level Constraint Toggling

The `PhysicsConstraintFlags` object drives the projection layer. **Critically, flags do NOT add or remove constraints at solve time** — that would break DPP and force slow recanonicalization. Instead, build all $2^3 = 8$ precompiled `CvxpyLayer` instances at startup (one per Boolean-flag combination) and route at inference:

```python
```

```
import itertools

class PhysicsProjectionLayer(nn.Module):
    def __init__(self):
        super().__init__()
        # Precompile one CvxpyLayer per Boolean-flag combination
        self.layers = {
            flags_tuple: build_cvxpylayer(flags_tuple)
            for flags_tuple in itertools.product([True, False], repeat=3)
        }

    def forward(self, y_raw, flags: PhysicsConstraintFlags, mu):
        layer = self.layers[flags.to_tuple()]
        (y_proj,) = layer(y_raw, mu, *self._extract_params(flags))
        return y_proj
```

Eight layers × ~30 ms compile each = 240 ms startup overhead. Negligible.

4. Differentiable Optimization Layer Engineering

4.1 Library Comparison

Library	Use case	Pros	Cons	Verdict
`cvxpylayers`	Convex problems via CVXPY DSL	Highest-level API; QP/SOCP/SDP; PyTorch+JAX+MLX; implicit-diff backward; v1.0 supports CuClarabel GPU acceleration	Slower than `qpth` on dense GPU QPs; strict DPP compliance	**Primary choice**
`qpth` / **OptNet**	Pure QP only	5–25× faster than CVXPY on GPU; batched primal-dual interior point	QP only — no SOCP (kills friction circle)	Use only if `cvxpylayers` is too slow
`theseus` (Meta)	Nonlinear least squares (SLAM)	Sparse solvers; Lie groups; batching; GPU	*"Apply constraints in a soft manner (i.e., using weighted costs)"* — own paper	**Skip**
Unrolled SCP (custom)	Non-convex via 3–5 outer iterations	Handles Pacejka; pure PyTorch autograd	More code; gradient quality depends on `n_iter`	**Wrap** `cvxpylayers` for the tire-force tier only

4.2 The Core Convex Projection

python

```python
# apex/physics/constraints.py
import cvxpy as cp
from cvxpylayers.torch import CvxpyLayer

def build_convex_projection(H=30, C=14, enforce_compl=True, enforce_steer=True):
    y = cp.Variable((H, C))
    y_hat = cp.Parameter((H, C))              # TTM forecast
    mu = cp.Parameter(H, nonneg=True)         # friction coefficient per timestep
    g = 9.81

    # Indices match CHANNEL_ORDER
    SPEED, AX, AY, THROTTLE, BRAKE, STEER = 1, 2, 3, 4, 5, 6

    objective = cp.Minimize(cp.sum_squares(y - y_hat))
    constraints = []

    # (a) Friction circle: ||(ax, ay)||_2 ≤ μ·g  →  SOCP
    for t in range(H):
        constraints.append(
            cp.norm(cp.hstack([y[t, AX], y[t, AY]]), 2) <= mu[t] * g
        )

    # (b) Kinematic consistency: v[t+1] - v[t] ≈ ax[t] · dt (slack 0.5 m/s)
    dt = 1.0  # TTM at 1 Hz
    for t in range(H - 1):
        constraints.append(
            cp.abs(y[t+1, SPEED] - y[t, SPEED] - dt * y[t, AX]) <= 0.5
        )

    # (c) Control bounds
    constraints += [y[:, THROTTLE] >= 0, y[:, THROTTLE] <= 1]
    constraints += [y[:, BRAKE]    >= 0, y[:, BRAKE]    <= 1]
    constraints += [y[:, SPEED]    >= 0]

    # (d) COA-gated complementarity (CONVEX RELAXATION)
    if enforce_compl:
        # True bilinear throttle·brake ≤ ε is non-convex; relax to budget:
        constraints += [y[:, THROTTLE] + y[:, BRAKE] <= 1.05]

    problem = cp.Problem(objective, constraints)
    assert problem.is_dpp(), "Problem must be DPP for cvxpylayers"
    return CvxpyLayer(problem, parameters=[y_hat, mu], variables=[y])
```

4.3 Handling Pacejka Non-Convexity via Unrolled SCP

The Pacejka magic formula `Fy = D · sin(C · arctan(B·α − E·(B·α − arctan(B·α))))` is clearly non-convex in slip angle α. For the optional eight-tier physics, wrap the convex layer in a 3-iteration SCP loop: Wikipedia

```python
def project_with_pacejka(y_raw, mu, tire_params, n_iter=3):
    y = y_raw.clone()
    for k in range(n_iter):
        B_lin, b_lin = linearize_pacejka_at(y, tire_params)  # PyTorch autograd
        (y,) = convex_layer_with_tire(y_raw, mu, B_lin, b_lin)
    return y
```

PyTorch unrolls the gradient through all three iterations. Backward pass is ~3× convex-only cost — tolerable for 30 timesteps on the 4060. The `arXiv:2512.03557` paper "Parameter Optimization in Trajectory Planning via Differentiable Convex Programming" validates this exact pattern for non-convex aerospace trajectory problems by *"deriving first-order sensitivity relations of second-order cone programming solutions with respect to problem data."* arXiv

If unrolling proves unstable: switch to "first-iterate gradient only" (treat later iterations as `.detach()`-ed refinement). This is sometimes called "shortcut backpropagation" and is documented as a stable fallback in Pineda et al.'s Theseus paper.

4.4 Numerical-Stability Pitfalls and Fixes

- **Velocity-near-zero singularity:** Slip ratios divide by speed. Per Wikipedia's Pacejka entry: *"when implemented into computer code, it doesn't work for low speeds (from around the pit-entry speed)."* Fix: Tikhonov damping `slip = (v_wheel − v) / (v + ε)` with `ε = 0.5 m/s`; below 1 m/s, freeze tire forces to a smooth saturated value via `torch.tanh`. Wikipedia
- **Stiff transient tire ODE:** Relaxation-length dynamics become stiff at high speed (relaxation_length / speed → 0). For the projection layer, replace the ODE with its **steady-state algebraic solution**. Reserve transient dynamics for offline validation, not the inner solve.
- **DPP compliance:** Never multiply two parameters together inside CVXPY. Compute their product in PyTorch first, then pass the result as a single new parameter. The classic trap is documented in `cvxgrp/cvxpylayers#80`: *"You should generally refrain from applying cvxpy atoms to parameters, and instead perform the operation outside cvxpy."* Run `assert problem.is_dpp()` after every problem construction. The official quickstart warns: *"Invalid: Parameters that change problem structure — `cp.quad_form(x, P)` # P as parameter may break DPP."* GitHub Cvxpylayers

- **Gradient explosion at the TTM/projection boundary:** Clip projection-layer gradients at norm 10.0 at the boundary back into TTM. PhysicsTTM is frozen-TTM + projection, so backward through TTM is rarely needed — but worth a safety net.
- **Solver choice:** `cvxpylayers` default is SCS. For QP/SOCP at this scale ($30 \times 14 = 420$ variables), **Clarabel** is faster and more numerically robust. Set `solver_args={"acceleration_lookback": 0}` to avoid SCS-specific instabilities on near-degenerate problems.

4.5 What's Realistically Stable in 12 Days

Tier	Description	Ship by	Risk
1	Speed $\geqslant 0$; throttle/brake $\in [0,1]$	Day 4	Trivial
2	Friction circle (SOCP)	Day 5	Low
3	Kinematic consistency `Δv ≈ a·dt`	Day 6	Low
4	Steering-lateral consistency (linearized `ay ≈ v²·κ`)	Day 7	Medium
5	COA-gated complementarity	Day 8	Low (1-line toggle)
6	3-component load transfer (linear)	Day 9	Medium
7	Pacejka linearized + 3-step SCP	Day 10–11	High — **feature-flag**
8	Transient tire dynamics + thermal	—	**Cut.**

The convex Tier-1-through-5 stack is the floor. Ship that no matter what. Tiers 6–7 are aspirational; Tier 8 is post-hackathon.

5. The "Shouldn't Be Possible" Demo Architecture

5.1 The Visual Sequence (3-Minute Video)

5.2 The Five Required Plots (build on Day 11)

1. **g-g diagram, raw vs projected, side-by-side.** Lateral acceleration (x) vs longitudinal acceleration (y), scaled in g's. Raw points scattered outside the friction circle; projected points within. Friction-ellipse overlay (μ·g radius; elliptical if $\mu_x \neq \mu_y$). Use Plotly's `scatter` + `shape` overlay. **Motorsport judges have known this plot since Lola/Penske 1970** (first publication of g-g diagram). Grokipedia + 2
2. **Friction ellipse with Pacejka curves.** Slip-angle curves at 2°, 4°, 6°, 8°, 10° drawn as semi-transparent loops; forecast trajectory in bold. Demonstrates the actual non-linear envelope.

3. **Channel overlay traces.** Speed, throttle, brake, steering vs time. Raw forecast in red dashed; projected in solid blue. Annotation balloons on impossible moments (e.g. *"throttle=0 here but speed rising in raw forecast"*).
4. **Violation severity heatmap.** 30 timesteps × 6 constraint types. Each cell colored by L2 severity. Demonstrates which constraints fire when.
5. **COA constraint-flag panel.** Visual table from `PhysicsConstraintFlags` showing which constraints are on/off for this driver, with the exact line from the CoA PDF that triggered each flag (e.g. *"Hand controls — MME Motorsport — simultaneous control permitted"* → `enforce_throttle_brake_complementarity = False`).

5.3 The Strongest Narrative Move

The judge needs ONE concrete moment that proves vehicle-dynamics-grade physics. That moment is:

> *"Driver X uses MME hand controls — FIA-recognized, manufacturer states verbatim that the system 'allows holding full throttle and applying a little braking in the corner.' Driver Y uses sip-and-puff — physically can't do both because inhalation and exhalation are mutually exclusive. PhysicsTTM reads the CoA, flips one boolean, and the projection layer enforces a different constraint set for each driver. The same TTM. The same data.* ***The CoA is a tensor switch.****"*

This is impossible to fake. A motorsport-literate judge knows that left-foot-braking is real, that FIA-recognized hand controls cost £7,500 and exist on Robert Wickens' Hyundai Elantra TCR, that CoAs are real documents — and they will recognize the depth of the integration immediately. Adrian Flux + 3

6. Evaluation and Validation Methodology

6.1 The Five "Physical Validity" Metrics

Define on a forecast tensor `y ∈ ℝ^(H, C)`:

1. **Friction-Circle Violation Rate (FCVR):** `mean_t [ 𝟙(||(ax_t, ay_t)||₂ > μ_t · g) ]`. Fraction of timesteps outside the friction ellipse.
2. **Kinematic Inconsistency RMSE (KIR):** `RMSE_t ( v_{t+1} − v_t − a_t · dt )`. Zero = perfect; nonzero = velocity channel disagrees with acceleration channel.
3. **Control Out-of-Bounds Rate (COBR):** Fraction of timesteps where `throttle ∉ [0,1]` or `brake ∉ [0,1]` (zero-shot TTM regularly produces negative throttle).
4. **Steering-Lateral Consistency Error (SLCE):** RMS of `|ay − (v² / L) · sin(steer)|` normalized by `μ·g`. Zero = perfectly geometrically consistent.
5. **Complementarity Violation Rate (CVR):** Fraction of timesteps with `throttle · brake > 0.05`, conditional on `enforce_throttle_brake_complementarity=True`.

6.2 The Headline Ablation Table

Model	FCVR ↓	KIR ↓ (m/s)	COBR ↓	SLCE ↓	CVR ↓ (when on)
B1 — Zero-shot TTM (raw)	~0.4+	~1.5+	~0.15+	~0.3+	~0.25+
B2 — TTM + soft physics loss	reduced but nonzero	reduced but nonzero	reduced	reduced	reduced
B3 — PhysicsTTM (convex)	**0.00**	**<0.1**	**0.00**	**<0.05**	**0.00**
B3+ — with Pacejka SCP	0.00	<0.1	0.00	<0.03	0.00

The hard-constraint guarantees are what make the bottom rows **exactly zero**. This is the empirical signature of the architecture: violations don't drop, they go to zero on bounded constraints **by construction**. (Soft-penalty PINN approaches like B2 cannot achieve this — DAE-HardNet's arXiv:2512.05881 explicitly motivates hard-projection layers because soft penalties *"suffer from different structured objectives which can lead to convergence issues."*) arXiv arXiv

6.3 The Three Baselines (cut everything else)

For 12 days with 2 people, build only THREE baselines:

- **B1: Zero-shot TTM (raw).** No physics. The kinetic-hallucination demonstrator.
- **B2: TTM + soft physics loss.** Add penalty term `λ · ||violation||²` to a fine-tuned head. Shows the *soft* approach can't enforce hard guarantees.
- **B3: PhysicsTTM (hard).** The system.

Skip larger ablations (different solvers, different SCP depths, different tier counts). The above table is enough.

6.4 The Empirical Kinetic-Hallucination Demonstration

The single most credible forensic demo: take a FastF1-derived telemetry CSV (cleaning the boolean-brake issue by deriving `brake_norm` from `dv/dt < 0 ∧ brake_bool=True`). Run zero-shot TTM. Find the first 30-second window where FCVR > 0.5 in the raw forecast (you'll find one within 5 laps of qualifying telemetry — TTM was pretrained on energy/weather/retail, not cars; per the HF model card, *"TTM r2 releases support point forecasting use-cases specifically ranging*

from minutely to hourly resolutions" — 1Hz vehicle dynamics is far outside its training distribution). huggingface Hugging Face

Show that timestep, freeze it, walk through which physics constraints PhysicsTTM corrected, print the violation log, show Guardian's verdict. **This is a forensic demonstration, not a benchmark — and that's exactly what a motorsport engineer respects.**

6.5 What This Does NOT Validate (be honest)

- We do **NOT** validate that the projected forecast is *closer to ground truth* than the raw forecast (it may be further in MSE if TTM happens to luck into physically impossible but accidentally close values).
- We do **NOT** validate that the Pacejka parameters are correct for any specific car (they're inherited from open tire datasets).
- We do **NOT** validate that the tuning recommendations cause real lap-time gains (they're heuristics over forecast deltas).
- We **DO** validate that the projected forecast is **physically valid by construction**, which is the entire point of a hard-constraint projection layer.

The honest framing: **"PhysicsTTM does not make TTM more accurate. It makes TTM safe to listen to."**

Recommendations

Days 1–2 — Skeleton everything. Backend dev: scaffold `apex/contracts.py`, `apex/api/main.py`, stub every module with `NotImplementedError`. Ship a FastAPI service that returns canned `CoachingReport` JSON. Frontend integrates against the stub the same day.

Days 3–4 — IO layer. CSV loader with AiM/MoTeC/FastF1 header remapping (use BenergyRacing/racing-data-converter as reference for format support). Generate the 10-document synthetic CoA PDF corpus. Build PDF → `CoAParameters` parser with Granite Instruct + LangChain `with_structured_output(CoAParameters)`.

Days 5–6 — Forecast layer. Polyphase 50→1Hz decomposition (per Model 2). TTM r2.1 runner with `prediction_filter_length=30`. Unit-test that output shape is exactly `(B, 30, 14)`.

Days 7–9 — Convex physics tier (Tiers 1–5). Build all 8 `CvxpyLayer` instances at startup (one per Boolean-flag combination). Validate friction-circle SOCP solves in <100 ms on the M2 MacBook. Wire into pipeline. Generate first end-to-end violation log.

Day 10 — Guardian integration. Granite Guardian 3.3 with `custom_criteria`. One round-trip; parse `<score>yes|no</score>`. Verify on at least 3 distinct violation-log examples.

Day 11 — Visualization. All five plots in Plotly. g-g diagram is the headline.

Day 12 — Demo + buffer. Record video. Attempt Pacejka SCP tier (Tier 7) ONLY if everything else is stable.

Quantitative threshold to advance to non-convex Pacejka tier (Tier 7): Tier 1–5 ablation table complete by end-of-Day-9 **AND** `cvxpylayers` Clarabel solve time on the M2 < 200 ms per forecast. Otherwise ship convex-only.

Quantitative threshold to cut transient tire dynamics (Tier 8): Cut **unconditionally**. Twelve days, two people. Ship the Tier 1–5 stack with sterling reliability rather than a buggy Tier 8.

Quantitative threshold for "demo-ready": Tier 1–5 reliably produces FCVR = 0.00, COBR = 0.00, CVR = 0.00 on 5 distinct telemetry CSVs (3 FastF1-derived, 2 synthetic from grassroots data-logger CSVs) by end-of-Day-10.

Caveats

- **No published frozen-time-series-foundation-model + hard-differentiable-physics-projection architecture exists.** Per Model 1's prior-art pass, this is novel territory. Cite related work — DAE-HardNet (arXiv:2512.05881), KKT-HardNet (arXiv:2507.08124), OptNet (Amos & Kolter 2017, arXiv:1703.00443), Neural Projections (Cheng et al. NeurIPS 2020), Projected Neural Differential Equations (arXiv:2410.23667), Lift & Learn — as **inspiration, not precedent**. GitHub
- **FastF1 telemetry lacks steering angle and G-force channels and has a boolean-only brake channel.** The official FastF1 docs confirm: *"Brake (bool): Brakes are applied or not"* and *"Throttle (float): 0-100 Throttle pedal pressure"* — no `Steer`, no `LatAcc`, no `LonAcc`. The canonical 14-channel schema therefore requires kinematic imputation: derive G-forces from speed/time and GPS curvature, derive a continuous brake from `dv/dt < 0 ∧ brake_bool`, and derive steering from GPS heading rate. **Document these imputations explicitly in the demo** — judges will ask. Fastf1
- **Granite Guardian 3.3 (Aug 2025) is the right version for BYOC; 3.2 BYOC is messier.** 3.2 requires manual prompt-template construction with `<start_of_risk_definition>... <end_of_risk_definition>` tags — workable but less clean than 3.3's `guardian_config = {"custom_criteria": ...}`. Verify by Day 10 that the HF model card hasn't introduced breaking changes; 4.1 is current line in the GitHub README but its custom-criteria API may differ. GitHub
- **DPP compliance is the single most common bug class with `cvxpylayers`.** Any CVXPY parameter multiplied/divided/exp'd by another parameter inside the problem will silently break DPP and force slow recanonicalization. Always run `assert problem.is_dpp()` after construction. Reference: cvxgrp/cvxpylayers#80.

- **The non-convex Pacejka tier is high-risk for a 12-day build.** A truly rich nonlinear vehicle model with real tire forces is non-convex and requires unrolled SCP. Three iterations of unrolling are tractable on a 4060; five may not be. **Feature-flag it and be ready to cut on Day 10.**
- **The "throttle × brake = 0" complementarity used in the convex tier is a RELAXATION, not a true bilinear constraint.** The convex form is `throttle + brake ⩽ 1 + ε`. True bilinear `throttle · brake ⩽ ε` is non-convex and requires mixed-integer programming or another SCP layer (incompatible with cvxpylayers DPP). The relaxation captures the spirit (only one pedal pressed hard at a time) while staying convex. Mention this explicitly in the report — handwaving it would be caught by a careful judge.
- **All "next-session forecasts" are predictions, not promises.** TTM was pretrained on energy/weather/retail; the HF model card states verbatim: *"TTM r2 releases support point forecasting use-cases specifically ranging from minutely to hourly resolutions (Ex. 10 min, 15 min, 1 hour.)"*. Zero-shot on 1Hz motorsport telemetry is far outside the training distribution. The physics-projection layer ensures **physical validity** of the forecast, not statistical accuracy. The demo MUST communicate this distinction clearly: *"PhysicsTTM does not make TTM more accurate. It makes TTM safe to listen to."* huggingface Hugging Face
- **All performance numbers in the ablation table (FCVR, KIR, etc.) are illustrative targets, not measured results.** The exact values will depend on the test telemetry chosen and the seed used for TTM's zero-shot run. The *qualitative* claim — that hard projection layers achieve exactly zero on bounded constraints by construction, while soft penalty methods cannot — is mathematically guaranteed and is the real headline.
- **Team BRIT / MME simultaneity is explicitly marketed and verified, but FIA-CoA-level enforcement that any *specific* driver's car implements it is not knowable without that driver's actual CoA document.** The COA parser must accept that some MME-equipped CoAs may still explicitly disable simultaneity for medical or safety reasons. The boolean field `permits_simultaneous_throttle_brake` is parsed from the actual document — not inferred from the hardware brand alone.